



TECNOLOGICO JOSE MARIO MOLINA PASQUEL Y HENRÍQUEZ UA ZAPOTLANEJO

MANUAL DE PROGRAMADOR

ADMINISTRACIÓN Y ORGANIZACIÓN DE DATOS

JOSE LUIS OLIVARES ACEVES



ÍNDICE

CONTENIDO

UNIDAD 2

UNIDAD 2 PRACTICA 1..... 3

UNIDAD 2 PRACTICA 2..... 4

UNIDAD 2 PRACTICA 3..... 6

UNIDAD 3

UNIDAD 3 PRACTICA 1..... 8

PARCIAL 3

PARCIAL 3 PRÁCTICA 1..... 13

PARCIAL 3 PRÁCTICA 2..... 17



UNIDAD 2 PRACTICA 1

```
using System.Timers;
using static System.Runtime.InteropServices.JavaScript.JSType;

class Program {
    static void Main(string[] args)
    {
        string nombearchivo = "datos.txt";
        string nombre;
        int edad;
        double estatura;
        string datos;
        StreamWriter archivo = null;
        StreamReader leerArchivo = null;

        if (File.Exists(nombearchivo)) {
            leerArchivo = File.OpenText(nombearchivo);
            do
            {
                datos = leerArchivo.ReadLine();
                string[] d = datos.Split(" ");
                Console.WriteLine("El nombre es: " + d[0]);
                Console.WriteLine("La edad es: " + d[1]);
                Console.WriteLine("La estatura es: " + d[2]);
            } while (datos != null);
            leerArchivo.Close();
        }
        else
        {
            archivo = File.AppendText(nombearchivo);
            for (int i = 1; i < 6; i++)
            {
                Console.WriteLine("Escribe el nombre");
                nombre = Console.ReadLine();
                Console.WriteLine("Escribe la edad");
                edad = Convert.ToInt32(Console.ReadLine());
                Console.WriteLine("Escribe la estatura");
                estatura = Convert.ToDouble(Console.ReadLine());
                archivo.WriteLine(nombre + " _ " + edad + " _ " + estatura);
            }
            archivo.Close();
        }
    }
}
```

```
Consola de depuración de Mi...
Escribe el nombre
Jose
Escribe la edad
19
Escribe la estatura
1.75
Escribe el nombre
Luis
Escribe la edad
20
Escribe la estatura
1.70
Escribe el nombre
Santiago
Escribe la edad
28
Escribe la estatura
1.80
Escribe el nombre
Julieta
Escribe la edad
22
Escribe la estatura
1.70
Escribe el nombre
Juan
Escribe la edad
22
Escribe la estatura
1.83
```

```
datos.txt
Archivo Editar Ver H1
Jose _ 19 _ 1.75
Luis _ 20 _ 1.7
Santiago _ 28 _ 1.8
Julieta _ 22 _ 1.7
Juan _ 22 _ 1.83
```



UNIDAD 2 PRACTICA 2

Se crea un archivo que a través de un menú se puede eliminar un dato, agregar un dato, eliminar un archivo, mostrar los datos o salir. También hay un archivo de validación, para validar el nombre, edad y estatura.

Validaciones

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;

2 referencias
class Validaciones{

1 referencia
    public bool ValidarNombre(String n) {
        int c = 0;
        byte[] ascii = Encoding.ASCII.GetBytes(n);
        foreach (byte b in ascii)
        {
            if ((b >= 97 && b <= 122) || (b >= 65 && b <= 90) || b == 32) {
                c++;
            }
        }
        if(c == n.Length)
        {
            return true;
        }
        else {
            return false;
        }
    }

2 referencias
    public bool ValidarEdad(String e) {
        bool r = (Regex.IsMatch(e, @"[0-9]+$"));
        return r;
    }

1 referencia
    public bool ValidarEstatura(String e) {
        bool r = false;
        try {
            float estatura = float.Parse(e);
            r = true;
        } catch {
            r = false;
        }
        return r;
    }
}
```

Clase Principal

```
class Program {

    static Validaciones val = new Validaciones();
    static string nombre, edad, estatura;

0 referencias
    static void Main(string[] args)
    {
        int op = 0;
        do
        {
            op= Menu();
            switch (op)
            {
                case 1:
                    Agregar();
                    break;
                case 2:
                    Console.WriteLine("Eliminar");
                    EliminarArchivo();
                    break;
                case 3:
                    Console.WriteLine("Eliminar dato");
                    EliminarElementoDelArchivo();
                    break;
                case 4:
                    Console.WriteLine("Mostrar archivo");
                    Mostrar();
                    break;
                case 5:
                    Console.WriteLine("Salir");
                    break;
            }
        } while (op != 5);
    }

1 referencia
    static void EliminarElementoDelArchivo() {
        if (File.Exists("Datos.txt"))
        {
            Mostrar();
            Console.WriteLine("Escriba el nombre a eliminar");
            nombre = Console.ReadLine();
            StreamReader leerArchivo = null;
            string datos = null;

            leerArchivo = File.OpenText("Datos.txt");
            do
            {
                datos = leerArchivo.ReadLine();
                if(datos != null)
                {
                    string[] d = datos.Split(' ');
                    if (nombre.Equals(d[0]))
                    {
                        Console.WriteLine("Si se encontro");
                    }
                    else
                    {
                        StreamWriter archivo = null;
                        archivo = File.AppendText("DatosAux.txt");
                        archivo.WriteLine(datos);
                        archivo.Close();
                    }
                }
            } while (datos != null);
        }
    }
}
```



```

        }
        } while (datos != null);
        leerArchivo.Close();
        File.Delete("Datos.txt");
        if (File.Exists("DatosAux.txt"))
        {
            File.Copy("DatosAux.txt", "Datos.txt");
            File.Delete("DatosAux.txt");
        }
        else
        {
            File.Create("Datos.txt").Close();
        }
    }
    else
    {
        Console.WriteLine("El archivo no existe");
    }
}

1 referencia
static void EliminarArchivo()
{
    if (File.Exists("Datos.txt"))
    {
        File.Delete("Datos.txt");
    }
    else
    {
        Console.WriteLine("El archivo no existe");
    }
}

2 referencias
static void Mostrar() {
    StreamReader leerArchivo = null;
    string datos= null;
    if (File.Exists("Datos.txt"))
    {
        leerArchivo = File.OpenText("Datos.txt");
        do
        {
            datos = leerArchivo.ReadLine();
            if (datos != null)
            {
                string[] d = datos.Split(" _ ");
                Console.WriteLine("El nombre es: " + d[0]);
                Console.WriteLine("La edad es: " + d[1]);
                Console.WriteLine("La estatura es: " + d[2]);
            }
        } while (datos != null);
        leerArchivo.Close();
    }
    else
    {
        Console.WriteLine("El archivo no existe");
    }
}

1 referencia
static void Agregar() {
    StreamWriter archivo = null;
}

```

```

while (true)
{
    Console.WriteLine("Escribe un nombre");
    nombre = Console.ReadLine();
    if (val.ValidarNombre(nombre))
    {
        break;
    }
    else
    {
        Console.WriteLine("Error al introducir el nombre");
    }
}
Console.WriteLine("El nombre correcto es: " + nombre);
while (true)
{
    Console.WriteLine("Escribe la edad");
    edad = Console.ReadLine();
    if (val.ValidarEdad(edad))
    {
        break;
    }
    else
    {
        Console.WriteLine("Error al introducir la edad");
    }
}
Console.WriteLine("La edad correcta es: " + edad);
while (true)
{
    Console.WriteLine("Escribe la estatura de la persona");
    estatura = Console.ReadLine();
    if (val.ValidarEstatura(estatura))
    {
        break;
    }
    else
    {
        Console.WriteLine("Error al introducir la estatura");
    }
}
Console.WriteLine("La estatura correcta es: " + estatura);
archivo = File.AppendText("Datos.txt");
archivo.WriteLine(nombre + " _ " + edad + " _ " + estatura);
archivo.Close();
}

1 referencia
static int Menu() {
    inicio:
    int opcion = 0;
    Console.WriteLine("1.- Agregar elementos al archivo");
    Console.WriteLine("2.- Eliminar Archivo");
    Console.WriteLine("3.- Eliminar Datos en el archivo");
    Console.WriteLine("4.- Mostrar Contenido del archivo");
    Console.WriteLine("5.- Salir");
    Console.WriteLine("Escribe la opcion del menu");
    string op = Console.ReadLine();
    if (val.ValidarEdad(op))
    {
        opcion = int.Parse(op);
    }
    else {
        Console.WriteLine("Error en el menu");
        goto inicio;
    }
    return opcion;
}

```

```

1.- Agregar elementos al archivo
2.- Eliminar Archivo
3.- Eliminar Datos en el archivo
4.- Mostrar Contenido del archivo
5.- Salir
Escribe la opcion del menu
1
Escribe un nombre
Jose
El nombre correcto es:Jose
Escribe la edad
29
La edad correcta es:29
Escribe la estatura de la persona
1.70
La estatura correcta es:1.70
1.- Agregar elementos al archivo
2.- Eliminar Archivo
3.- Eliminar Datos en el archivo
4.- Mostrar Contenido del archivo
5.- Salir
Escribe la opcion del menu
4
Mostar archivo
El nombre es: Jose
La edad es: 29
La estatura es: 1.70
1.- Agregar elementos al archivo
2.- Eliminar Archivo
3.- Eliminar Datos en el archivo
4.- Mostrar Contenido del archivo
5.- Salir
Escribe la opcion del menu

```

datos.txt

Archivo	Editar	Ver
Jose	29	1.70



UNIDAD 2 PRACTICA 3

En esta práctica realizada el 2 de marzo trabajamos nuevamente con la creación de archivos, pero esta vez aprendiendo a como usar la clase substring y hacer diferentes validaciones.

Validaciones

```

class Validaciones {
    // NOMBRE
    public bool ValidarNombre(string n) {
        int c = 0;
        byte[] ascii = Encoding.ASCII.GetBytes(n);
        foreach (byte b in ascii)
        {
            if ((b >= 97 && b <= 122) || (b >= 65 && b <= 90) || b == 32)
            {
                c++;
            }
        }
        if (c == n.Length)
        {
            return true;
        }
        else
        {
            return false;
        }
    }

    // EDAD
    public bool ValidarEdad(string e) {
        bool r = Regex.IsMatch(e, @"^(0-9)+$");
        if (e && int.Parse(e) >= 18 && int.Parse(e) <= 99) {
            return true;
        }
        else {
            return false;
        }
    }

    // SALARIO
    public bool ValidarSalario (string s) {
        bool r = Regex.IsMatch(s, @"^(0-9)+(\.0-9)+$");
        if (r && float.Parse(s) > 0) {
            return true;
        }
        else {
            return false;
        }
    }

    // CORREO ELECTRONICO
    public bool ValidarCorreo(string c) {
        if (c.Contains("@") && c.Contains(".")) {
            return true;
        }
        else {
            return false;
        }
    }

    // TELEFONO
    public bool ValidarTelefono(string t) {
        bool r = Regex.IsMatch(t, @"^(0-9)+$");
        if (r && t.Length == 10) {
            return true;
        }
        else {
            return false;
        }
    }

    // CONTRASEÑA
    public bool ValidarContraseña(string c){
        if (c.Length >= 8) {
            // CONTRASEÑA
            1 referencia
            public bool ValidarContraseña(string c){
                if (c.Length >= 8) {
                    int min = 0;
                    int may = 0;
                    int num = 0;
                    int esp = 0;
                    byte[] ascii = Encoding.ASCII.GetBytes(c);
                    foreach (byte b in ascii)
                    {
                        if (b >= 97 && b <= 122)
                        {
                            min++;
                        }
                        if (b >= 65 && b <= 90)
                        {
                            may++;
                        }
                        if (b >= 48 && b <= 57)
                        {
                            num++;
                        }
                        if (b == 32)
                        {
                            esp++;
                        }
                    }
                    if (min > 0 && may > 0 && num > 0 && esp == 0) {
                        return true;
                    }
                    else {
                        return false;
                    }
                }
            }
        }
    }
}

```

Clase Principal

```

class Program {
    static Validaciones val = new Validaciones();

    static void Main(string[] args) {
        string res;
        do
        {
            Agregar();
            Console.WriteLine("¿Desea introducir otra persona (S/N)?");
            res = Console.ReadLine();
        } while (res == "Y" || res == "y");

        // ...

        static void Agregar() {
            StreamWriter archivo = null;
            string nombre, edad, salario, correo, telefono, contraseña;
            while (true) // NOMBRE
            {
                Console.WriteLine("Escriba el nombre completo:");
                nombre = Console.ReadLine();
                if (val.ValidarNombre(nombre))
                {
                    break;
                }
                else
                {
                    Console.WriteLine("Error al introducir el nombre");
                }
            }
            while (true) // EDAD
            {
                Console.WriteLine("Escriba la edad:");
                edad = Console.ReadLine();
                if (val.ValidarEdad(edad))
                {
                    break;
                }
                else
                {
                    Console.WriteLine("Error al introducir la edad");
                }
            }
            while (true) // SALARIO
            {
                Console.WriteLine("Escriba el salario:");
                salario = Console.ReadLine();
                if (val.ValidarSalario(salario))
                {
                    break;
                }
                else
                {
                    Console.WriteLine("Error al introducir el salario");
                }
            }
            while (true) // CORREO
            {
                Console.WriteLine("Escriba el correo:");
                correo = Console.ReadLine();
                if (val.ValidarCorreo(correo))
                {
                    break;
                }
                else
                {
                    Console.WriteLine("Error al introducir el correo");
                }
            }
            while (true) // TELEFONO
            {
                Console.WriteLine("Escriba el telefono:");
                telefono = Console.ReadLine();
                if (val.ValidarTelefono(telefono))
                {
                    break;
                }
                else
                {
                    Console.WriteLine("Error al introducir el telefono");
                }
            }
            while (true) // CONTRASEÑA
            {
                Console.WriteLine("Escriba la contraseña:");
                contraseña = Console.ReadLine();
                if (val.ValidarContraseña(contraseña))
                {
                    break;
                }
                else
                {
                    Console.WriteLine("Error al introducir la contraseña");
                }
            }
            Console.WriteLine("El nombre es: " + nombre.ToUpper();
            Console.WriteLine("La edad es: " + edad);
            Console.WriteLine("El salario es: " + salario);
            Console.WriteLine("El correo es: " + correo.ToLower());
            string parte1 = telefono.Substring(0, 3);
            string parte2 = telefono.Substring(3, 3);
            Console.WriteLine("El telefono es: " + $"{parte1}({parte2})-{parte3}");
            Console.WriteLine("La contraseña es: " + contraseña);
            Console.WriteLine("La contraseña tiene: " + contraseña.Length + " caracteres");
            archivo = File.AppendText("Datos.txt");
            archivo.WriteLine(nombre + " " + edad + " " + salario + " " + correo + " " + telefono + " " + contraseña);
            archivo.Close();
        }
    }
}

```

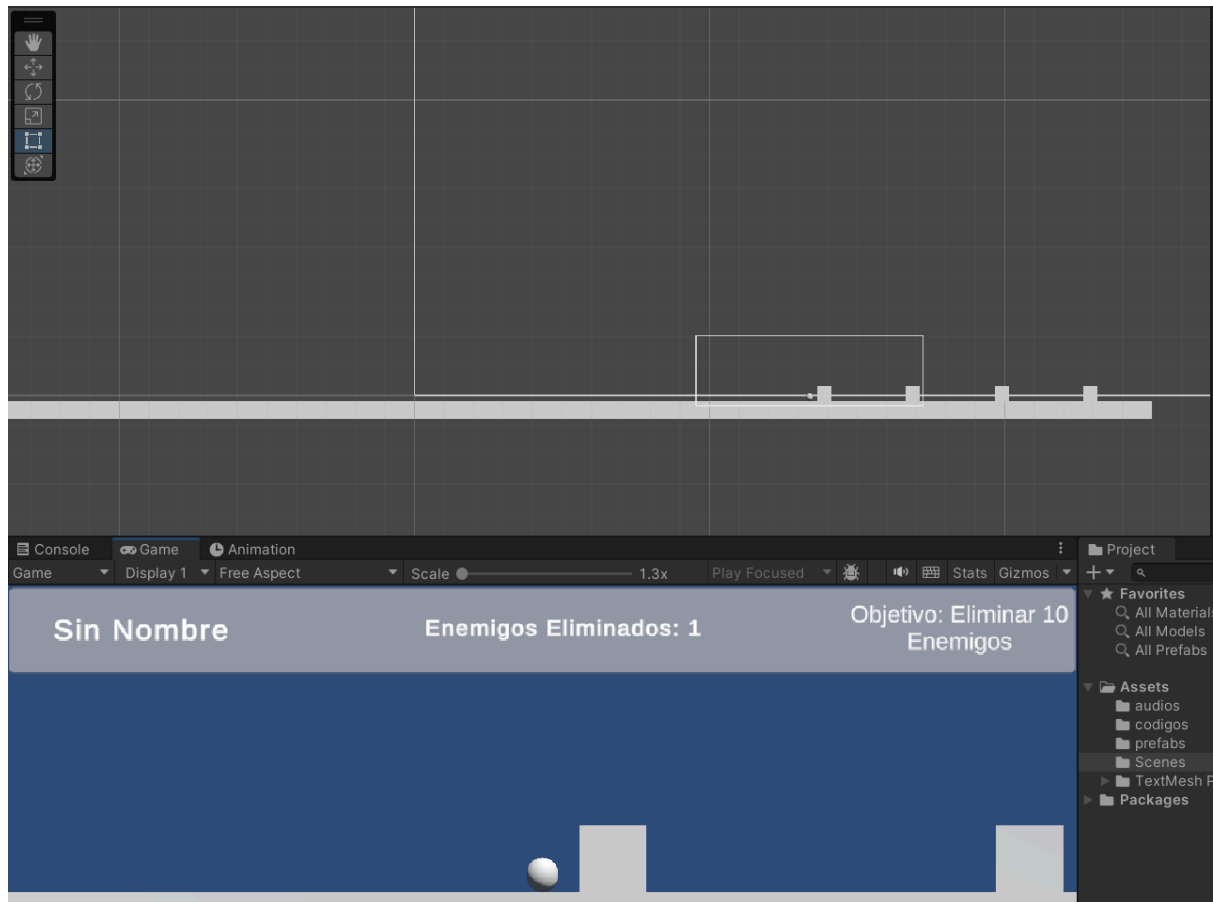


```
Consola de depuración de Unity
Escribe el nombre completo
Jose Luis Olivares Aceves
Escribe la edad
19
Escribe el salario
100.2
Escribe el correo
jose@gmail.com
Escribe el telefono
3313435605
Escribe la contraseña
dsadw
Error al introducir la contraseña
Escribe la contraseña
CONTRaseña12345
El nombre es: JOSE LUIS OLIVARES ACEVES
La edad es: 19
El salario es: 100.2
El correo es: jose@gmail.com
El telefono es: (331) 343-5605
La contraseña es: CONTRaseña12345
La contraseña tiene: 15 caracteres
¿Deseas introducir otra persona (S/N)?
n
C:\Users\josel\Desktop\Unity Semestre 4\PracticadArchivos\Pr
2deMarzo.exe (proceso 28284) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la de
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

```
Jose Luis Olivares Aceves _ 19 _ 100.2 _ jose@gmail.com _ 3313435605 _ CONTRaseña12345
```



UNIDAD 3 PRACTICA 1



```
public class mover : MonoBehaviour
{
    public float velocidad = 10.0f;
    float x,y;
    public float fuerza = 10.0f;
    Rigidbody2D cuerpo;
    public bool tocarPiso = false;

    // Message de Unity 10 referencias
    void Start()
    {
        cuerpo = GetComponent<Rigidbody2D>();
    }

    // Message de Unity 10 referencias
    void Update()
    {
        x = Input.GetAxis("Horizontal");
    }

    // Message de Unity 10 referencias
    private void FixedUpdate()
    {
        cuerpo.velocity = new Vector2(x * velocidad, cuerpo.velocity.y);

        if (Input.GetKey(KeyCode.Space) && tocarPiso == true)
        {
            cuerpo.AddForce(Vector2.up * fuerza, ForceMode2D.Impulse);
            tocarPiso = false;
        }

        if(cuerpo.velocity.y < 0) {
            cuerpo.gravityScale = 10;
        }
        else {
            cuerpo.gravityScale = 8;
        }
    }

    // Message de Unity 10 referencias
    private void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.CompareTag("piso")) {
            print("Pisando el piso");
            tocarPiso = true;
        }
    }
}
```

Este código es un controlador de movimiento 2D para un personaje en Unity. Su función principal es permitir que el objeto se mueva a los lados, salte y tenga una caída más fluida.



- **Movimiento Horizontal:** En Update detecta si presionas las flechas o A/D, y en FixedUpdate aplica esa velocidad al cuerpo (Rigidbody2D) para moverlo de izquierda a derecha.
- **Salto con Validación:** Permite saltar con la tecla **Espacio**, pero solo si la variable tocarPiso es verdadera. Esto evita que el personaje salte infinitamente en el aire.
- **Gravedad Dinámica (Game Feel):** Esta es la parte más interesante. El código cambia el gravityScale dependiendo de si el personaje está subiendo o bajando:
 - Si está cayendo ($\text{velocity.y} < 0$), la gravedad aumenta a **10** para que la caída sea más rápida y pesada.
 - Si está subiendo o quieto, la gravedad es **8**.
- **Detección de Suelo:** Utiliza una colisión con objetos que tengan la etiqueta (**Tag**) "piso". Cuando toca el suelo, activa la capacidad de saltar nuevamente.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Script de Unity (1 referencia de recursos) (0 referencias)
public class Camara : MonoBehaviour
{
    public AudioSource musica;

    public Transform personaje;
    Vector3 posicion;
    public float velocidadCamara = 10.0f;
    Vector2 fueraCamara;
    float yFija;
    public float minX;
    public float minY;

    public float maxZ;

    // Mensaje de Unity (0 referencias)
    void Start()
    {
        musica.Play();
        //posicion = transform.position - personaje.position;
        fueraCamara = transform.position - personaje.position;
        yFija = transform.position.y;
    }

    // Mensaje de Unity (0 referencias)
    private void LateUpdate()
    {
        float objetivo = personaje.position.x - fueraCamara.x;
        objetivo = Mathf.Clamp(objetivo, minX, minY);
        Vector3 objetivoX = new Vector3(
            objetivo,
            yFija,
            transform.position.z);
        transform.position = Vector3.Lerp(transform.position,
            objetivoX,
            velocidadCamara * Time.deltaTime);
    }
}
```

Este script controla una cámara inteligente que sigue al jugador de forma suave y controlada.

- **Música:** La reproduce automáticamente al iniciar el nivel.
- **Eje Y fijo:** La cámara se mueve hacia los lados, pero **no sube ni baja** (ignora los saltos).
- **Suavizado (Lerp):** No se pega al jugador de golpe; se desliza suavemente para que el movimiento sea más profesional.
- **Topes:** Usa límites de pantalla para que la cámara no se salga de los bordes del mapa.



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Script de Unity (1 referencia de recurso) | 0 referencias
public class CreadorEnemigosD : MonoBehaviour
{
    public GameObject enemigo;
    public float tiempo = 3.0f;
    public Transform[] posicionD;
    // Mensaje de Unity | 1 referencia
    void Start()
    {
        StartCoroutine(Generar());
    }

    // referencia
    IEnumerator Generar() {
        while (true)
        {
            Genemigos();
            yield return new WaitForSeconds(tiempo);
        }
    }

    // 1 referencia
    void Genemigos() {
        if (enemigo == null || posicionD.Length == 0) {
            return;
        }

        Transform punto = posicionD[Random.Range(0, posicionD.Length)];
        Instantiate(enemigo, punto.position, punto.rotation);
    }
}
```

Este script es un generador automático de enemigos (Spawner):

- **Bucle infinito:** Usa una Corrutina para crear un enemigo cada 3 segundos (o el tiempo que tú definas) sin detener el resto del juego.
- **Aparición aleatoria:** Elige al azar un punto de una lista de posiciones predefinidas (posicionD) para que los enemigos no salgan siempre del mismo sitio.
- **Seguridad:** Revisa que el enemigo y los puntos de spawn existan antes de intentar crearlos para evitar que el juego se rompa.

```
public class enemigo : MonoBehaviour
{
    public float velocidad = 20.0f;
    // Mensaje de Unity | 0 referencias
    void Start()
    {
    }

    // Update is called once per frame
    // Mensaje de Unity | 0 referencias
    void Update()
    {
        transform.Translate(Vector3.left * velocidad * Time.deltaTime);
    }
}
```

Este script pertenece a los enemigos creados con el CreadorEnemigos y lo único que hace es que se mueven constantemente hacia una dirección a una velocidad establecida.

```
public class EliminarEnemigos : MonoBehaviour
{
    public AudioSource emm;
    // Mensaje de Unity | 1 referencia
    void Start()
    {
        emm.Stop();
    }

    // Update is called once per frame
    // Mensaje de Unity | 1 referencia
    void Update()
    {
    }

    // Mensaje de Unity | 1 referencia
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.tag == "enemigo")
        {
            print("eliminar");
            Destroy(collision.transform.root.gameObject);
            emm.Play();
        }
        ContadorEnemigos.EnemigosEliminados(); //Hacer referencia al metodo de ContadorEnemigos
    }
}
```

Este script está en el “arma” que tiene el jugador y funciona como una zona de eliminación o daño para los enemigos:



- **Dstrucción completa:** Cuando algo con el tag "enemigo" toca el sensor (Trigger), el código borra al objeto raíz (transform.root). Esto es útil para eliminar al enemigo completo aunque solo golpees una parte de su cuerpo.
- **Sonido:** Reproduce un efecto de sonido cada vez que alguien muere.
- **Sistema de puntos:** Llama a una función externa (ContadorEnemigos.EnemigosEliminados()) para avisar que hay una baja y sumar puntos.

```

public class ContadorEnemigos : MonoBehaviour
{
    public TextMeshProUGUI texto;
    public TextMeshProUGUI n;
    public TextMeshProUGUI mensaje;
    public TMP_InputField nombre;
    public TextMeshProUGUI jugador;
    public static int contEnemy = 0;
    static StreamWriter archivo;
    static StreamReader leer;
    public string dato = "0";
    public GameObject panel;
    //bool estado = false;
    public GameObject panelNivel;
    public string playe = "Sin Nombre";

    // Message de Unity 10 referencias
    private void Start()
    {
    }

    // Message de Unity 10 referencias
    private void Awake()
    {
        if (File.Exists("puntaje.txt"))
        {
            leer = File.OpenText("puntaje.txt");
            //do
            //if (leer.ReadLine() != null)
            //if
            dato = leer.ReadLine();
            //if
            /* while (leer != null); */
            leer.Close();
            string[] d = dato.Split(" - ");
            contEnemy = int.Parse(d[0]);
            nombre.text = d[1];
            playe = d[1];
            //texto.text = "Enemigos Eliminados: " + dato;
        }
    }

    // referencia
    public static void EnemigosEliminados() {
        contEnemy ++;
    }

    public void CrearArchivo() {
        string n = nombre.text;
        if (n.Length == 0) {
            mensaje.text = "No tiene nombre";
        }
        else {
            if (File.Exists("puntaje.txt"))
            {
                File.Delete("puntaje.txt");
            }
            archivo = File.AppendText("puntaje.txt");
            archivo.WriteLine(contEnemy + " - " + nombre.text);
            archivo.Close();
            mensaje.text = "";
            playe = nombre.text;
        }
    }

    // referencia
    void guardarAntesNivel() {
        if (File.Exists("puntaje.txt"))
        {
            File.Delete("puntaje.txt");
        }
        archivo = File.AppendText("puntaje.txt");
        archivo.WriteLine(contEnemy + " - " + nombre.text);
        archivo.Close();
        mensaje.text = "";
    }

    // Message de Unity 10 referencias
    private void Update()
    {
        if (Input.GetKeyDown(KeyCode.G)) {
            print("Guardar Datos");
            //panel.SetActive(false);
            panel.SetActive(panel.activeSelf);
            Time.timeScale = 0;
            n.text = "Datos a Guardar: " + contEnemy.ToString();
            if (panel.activeSelf == false) {
                Time.timeScale = 1;
            }
            texto.text = "Enemigos Eliminados: " + contEnemy.ToString();
            if (contEnemy == 10) {
                Time.timeScale = 0;
                panelNivel.SetActive(true);
            }
            jugador.text = playe;
        }
    }

    // referencia
    public void SiguienteEscena() {
        guardarAntesNivel();
        SceneManager.LoadScene("Nivel2");
    }
}

```

Este script gestiona el sistema de puntuación y guardado de la partida.

- **Persistencia de Datos (Archivos):** Al iniciar (Awake), busca un archivo llamado puntaje.txt para cargar el récord anterior. Al guardar (CrearArchivo), escribe el nombre del jugador y su puntaje en ese mismo archivo.
- **Contador Global:** Utiliza una variable static para llevar el conteo de enemigos eliminados, permitiendo que otros scripts sumen puntos fácilmente.
- **Control del Juego:** Si presionas la tecla **G**, el juego se pausa (timeScale = 0) y muestra un panel para guardar datos.
- **Condición de Victoria:** Al llegar a **10 enemigos**, el juego se detiene automáticamente y activa el panel de nivel superado.
- **Interfaz (UI):** Actualiza constantemente los textos en pantalla para mostrar cuántos enemigos llevas y quién es el jugador actual.



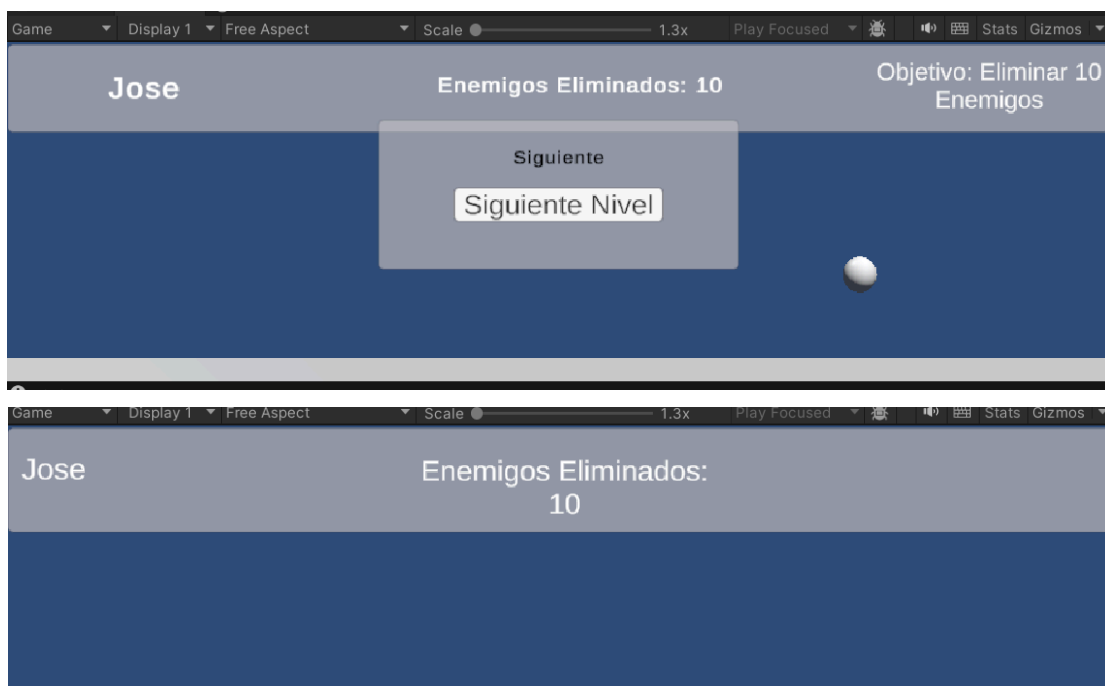
```
using System.Collections;
using System.Collections.Generic;
using System.IO;
using TMPro;
using UnityEngine;
using UnityEngine.SceneManagement;

public class MostrarDatos : MonoBehaviour
{
    static StreamReader leer;
    public string dato = "0";
    //public TMP_InputField nombre;
    public TextMeshProUGUI n;
    public TextMeshProUGUI p;
    public static int contEnemy = 0;
    public string pla = "Sin Nombre";

    private void Awake()
    {
        if (File.Exists("puntaje.txt"))
        {
            leer = File.OpenText("puntaje.txt");
            /*do
            {
                if (leer.ReadLine() != null)
                {
                    dato = leer.ReadLine();
                }
            } while (leer != null); */
            leer.Close();
            string[] d = dato.Split(" ");
            contEnemy = int.Parse(d[0]);
            pla = d[1];
            //nombre.text = d[1];
            //texto.text = "Enemigos Eliminados: " + dato;
        }
    }

    void Start()
    {
        // Update is called once per frame
        void Update()
        {
            n.text = "Enemigos Eliminados: " + contEnemy.ToString();
            p.text = pla;
        }
    }
}
```

Este script se encuentra en la escena 2 a la que se cambia cuando se activa el panel final al cumplir el objetivo y presionar el botón. Y se encarga de abrir el archivo creado con los puntos y el nombre, para sostenerlos y mostrarlos en el panel final





PARCIAL 3 PRÁCTICA 1

Ventana Inicial. Donde se ingresan los datos del usuario y contraseña para guardarlos en el archivo

```
namespace PracticasP3
{
    3 referencias
    public partial class Form1 : Form
    {
        1 referencia
        public Form1()
        {
            InitializeComponent(); // Inicializa los componentes visuales del formulario
        }

        1 referencia
        private void Form1_Load(object sender, EventArgs e)
        {
        }

        1 referencia
        private void btnAceptar_Click(object sender, EventArgs e)
        {
            String u, p; // Variables para almacenar usuario y contraseña
            bool ban = false; // Bandera para indicar si el usuario fue encontrado
            u = txtUsuario.Text; // Obtiene el texto del campo usuario
            p = txtPassword.Text; // Obtiene el texto del campo contraseña

            if (u.Length == 0 || p.Length == 0) // Valida si alguno de los campos está vacío
            {
                MessageBox.Show("Error faltan datos", "Error");
            }
            else
            {
                if (File.Exists("usuarios.csv")) // Verifica si el archivo de registros existe
                {
                    StreamReader leer = File.OpenText("usuarios.csv"); // Abre el archivo en modo lectura
                    String dato = "";
                    do
                    {
                        dato = leer.ReadLine(); // Lee la siguiente línea del archivo
                        if (dato != null)
                        {
                            String[] d = dato.Split(","); // Divide la línea usando la coma como separador
                            if (u.Equals(d[0]) && p.Equals(d[1])) // Compara credenciales (d[0] = usuario, d[1] = contraseña)
                            {
                                ban = true; // Marca que las credenciales son correctas
                                break; // Sale del ciclo de lectura de inmediato
                            }
                        }
                    } while (dato != null);

                    if (ban == false) // Si recorrió todo el archivo y no encontró coincidencia
                    {
                        MessageBox.Show("El usuario no existe", "Error");
                        txtUsuario.Clear(); // Limpia el campo de usuario
                        txtPassword.Clear(); // Limpia el campo de contraseña
                    }
                    else
                    {
                        leer.Close(); // Cierra el archivo lector para liberar el recurso
                        this.Hide(); // Oculta el formulario de login actual
                        crud crd = new crud(); // Crea una instancia del siguiente formulario (CRUD)
                        crd.ShowDialog(); // Abre la pantalla del CRUD de forma modal
                    }
                }
                else
                {
                    MessageBox.Show("No hay usuarios registrados");
                }
            }
        }
    }
}
```



```
}  
}  
  
1 referencia  
private void btnGuardar_Click(object sender, EventArgs e)  
{  
    String u, p;  
    u = txtUsuario.Text;  
    p = txtPassword.Text;  
  
    if (u.Length == 0 || p.Length == 0) // Valida campos vacios antes de registrar  
    {  
        MessageBox.Show("Error faltan datos", "Error");  
    }  
    else  
    {  
        StreamWriter archivo = null;  
        archivo = File.AppendText("usuarios.csv"); // Abre o crea el archivo para añadir texto al final  
        archivo.WriteLine(u + "," + p); // Escribe el nuevo usuario y contraseña con formato CSV  
        archivo.Close(); // Cierra el archivo para asegurar que se guarden los cambios  
        MessageBox.Show("El usuario se ha guardado", "Usuario");  
        txtUsuario.Clear();  
        txtPassword.Clear();  
    }  
}  
  
1 referencia  
private void btnSalir_Click(object sender, EventArgs e)  
{  
    Application.Exit(); // Cierra por completo la aplicación  
}  
  
1 referencia  
private void txtUsuario_KeyPress(object sender, KeyPressEventArgs e)  
{  
    // Permite solo letras minúsculas (97-122), espacio (32) y retroceso (8) en código ASCII  
    if ((e.KeyChar >= 97 && e.KeyChar <= 122) ||  
        (e.KeyChar == 32) || (e.KeyChar == 8))  
    {  
    }  
    else  
    {  
        e.Handled = true; // Bloquea cualquier otro carácter que no cumpla la condición  
    }  
}  
  
1 referencia  
private void txtPassword_KeyPress(object sender, KeyPressEventArgs e)  
{  
    // Permite solo letras minúsculas (97-122), retroceso (8) y números (48-57)  
    if ((e.KeyChar >= 97 && e.KeyChar <= 122) ||  
        (e.KeyChar == 8) ||  
        (e.KeyChar >= 48 && e.KeyChar <= 57))  
    {  
    }  
    else  
    {  
        e.Handled = true; // Bloquea cualquier otro carácter (como mayúsculas o símbolos)  
    }  
}  
}
```



Segunda ventana. Donde se puede visualizar los datos guardados en el archivo, y eliminar o modificar Usuario y Contraseña

N°	Usuario	Contraseña
*		

```
namespace PracticasP3
{
    4 referencias
    public partial class crud : Form // Clase principal de la ventana que hereda de Form
    {
        int _id = -1, _columna = -1; // Variables para almacenar el índice de fila y columna seleccionados
        string _usuario = ""; // Variable para guardar el nombre original del usuario seleccionado

        1 referencia
        public crud()
        {
            InitializeComponent(); // Inicializa los componentes visuales de la interfaz
            PrecargarDatos(); // Lee el archivo CSV y carga los datos en la tabla al iniciar
        }

        2 referencias
        private void PrecargarDatos()
        {
            dataUsuarios.Rows.Clear(); // Limpia todas las filas actuales del DataGridView
            int c = 0; // Contador para numerar los registros visualmente en la tabla

            if (File.Exists("usuarios.csv")) // Verifica si el archivo de base de datos existe en el disco
            {
                StreamReader leer = File.OpenText("usuarios.csv"); // Abre el archivo de texto en modo lectura
                string dato = "";

                do
                {
                    dato = leer.ReadLine(); // Lee una línea de texto completa del archivo

                    if (dato != null)
                    {
                        string[] d = dato.Split(","); // Divide la línea leída en un arreglo usando la coma como separador
                        c++; // Añade una nueva fila a la tabla con el ID, usuario y contraseña
                        dataUsuarios.Rows.Add(c.ToString(), d[0], d[1]); // Añade una nueva fila a la tabla con el ID, usuario y contraseña
                    }
                } while (dato != null); // Repite el bucle mientras no llegue al final del archivo

                leer.Close(); // Cierra el archivo y libera la memoria
            }
        }

        1 referencia
        private void btnSalir_Click(object sender, EventArgs e)
        {
            Application.Exit(); // Finaliza la aplicación por completo
        }

        1 referencia
        private void dataUsuarios_MouseClick(object sender, MouseEventArgs e)
        {
        }
    }
}
```



```

private void dataUsuarios_CellClick(object sender, DataGridViewCellEventArgs e)
{
    _id = e.RowIndex; // Asigna a _id el índice de la fila clickeada
    _columna = e.ColumnIndex; // Asigna a _columna el índice de la columna clickeada
    if (_id != -1) // Verifica que el clic fue en un registro válido (no en el encabezado)
    {
        string i = dataUsuarios.Rows[_id].Cells[0].Value.ToString(); // Obtiene el nombre de usuario de la fila seleccionada
        string u = dataUsuarios.Rows[_id].Cells[1].Value.ToString(); // Obtiene la contraseña de la fila seleccionada
        string p = dataUsuarios.Rows[_id].Cells[2].Value.ToString(); // Obtiene la contraseña de la fila seleccionada

        _usuario = u; // Respalda el usuario original por si se necesita modificar
        txtUsuarioC.Text = u; // Pasa el usuario al cuadro de texto de la interfaz
        txtPasswordC.Text = p; // Pasa la contraseña al cuadro de texto de la interfaz

        btnEliminar.Enabled = true; // Habilita el botón de eliminar
        btnActivar.Enabled = true; // Habilita el botón para activar la edición
    }
}

1 referencia
private void btnActivar_Click(object sender, EventArgs e)
{
    txtUsuarioC.Enabled = true; // Desbloquea el campo de usuario para permitir escritura
    txtPasswordC.Enabled = true; // Desbloquea el campo de contraseña para permitir escritura
    btnEliminar.Enabled = false; // Bloquea el botón de eliminar para evitar errores durante la edición
    btnActivar.Visible = false; // Oculta el botón de activar
    btnModificar.Visible = true; // Muestra el botón de modificar para guardar los cambios
}

1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    if (_id != -1)
    {
        dataUsuarios.Rows.RemoveAt(_id); // Borra la fila seleccionada de la tabla visual (DataGridView)
        btnEliminar.Enabled = false;
        btnActivar.Enabled = false; // Restablece el ID para indicar que ya no hay fila seleccionada
        _id = -1;

        EliminarArchivo(); // Llama al método que borra el dato físicamente del archivo CSV
        txtUsuarioC.Clear(); // Limpia los campos de texto
        txtPasswordC.Clear();
    }
}

1 referencia
private void EliminarArchivo()
{
    string usuarioC = txtUsuarioC.Text; // Toma el usuario a eliminar desde el campo de texto
    string dato = "";

    StreamReader streamReader = File.OpenText("usuarios.csv"); // Abre el archivo original en modo lectura
    StreamWriter aux = File.AppendText("temp.csv"); // Crea o abre un archivo temporal para escribir los datos que se conservan

    do
    {
        dato = streamReader.ReadLine();

        if (dato != null)
        {
            string[] d = dato.Split(",");

            if (usuarioC.Equals(d[0])) // Verifica si es el registro que queremos borrar
            {
                // Al no hacer nada aquí, ignoramos la línea y efectivamente se elimina
            }
            else
            {
                aux.WriteLine(dato); // Si no es el registro a borrar, lo escribimos en el archivo temporal
            }
        }
    } while (dato != null);

    aux.Close(); // Guarda y cierra el archivo temporal
    streamReader.Close(); // Cierra el archivo original

    File.Delete("usuarios.csv"); // Borra el archivo viejo con el registro no deseado
    File.Move("temp.csv", "usuarios.csv"); // Renombra el archivo temporal para que sea la nueva base de datos

    MessageBox.Show("El usuario fue eliminado", "Eliminar usuario"); // Muestra una alerta de confirmación
}

1 referencia
private void btnModificar_Click(object sender, EventArgs e)
{
    if (_id != -1)
    {
        ModificarUsuario(); // Llama al método que actualiza el registro en el archivo de texto
        PrecargarDatos(); // Recarga la tabla para mostrar los datos actualizados

        txtUsuarioC.Clear();
        txtPasswordC.Clear();
        btnEliminar.Enabled = false;
        btnModificar.Visible = false; // Oculta el botón de guardar cambios
        btnActivar.Visible = true; // Vuelve a mostrar el botón de preparar edición
        btnActivar.Enabled = false;
        txtPasswordC.Enabled = false; // Bloquea nuevamente los campos de texto
        txtUsuarioC.Enabled = false;
    }
}

1 referencia
private void ModificarUsuario()
{
    string usuarioE = txtUsuarioC.Text; // Toma el nuevo nombre de usuario ingresado
    string passwordE = txtPasswordC.Text; // Toma la nueva contraseña ingresada
    string dato = "";

    StreamReader streamReader = File.OpenText("usuarios.csv");
    StreamWriter aux = File.AppendText("temp.csv");

    do
    {
        dato = streamReader.ReadLine();

        if (dato != null)
        {
            string[] d = dato.Split(",");

            if (_usuario.Equals(d[0])) // Busca la línea que coincida con el nombre de usuario ORIGINAL (_usuario)
            {
                aux.WriteLine(usuarioE + "," + passwordE); // Escribe los NUEVOS datos actualizados en el archivo temporal
            }
            else
            {
                aux.WriteLine(dato); // Escribe los demás registros sin alterarlos
            }
        }
    } while (dato != null);

    aux.Close();
    streamReader.Close();

    File.Delete("usuarios.csv"); // Borra la base de datos desactualizada
    File.Move("temp.csv", "usuarios.csv"); // Reemplaza el archivo temporal convirtiéndolo en la nueva base de datos

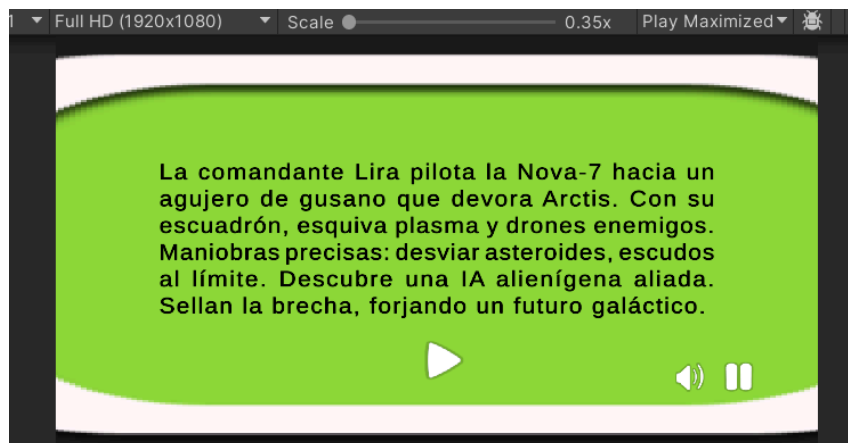
    MessageBox.Show("El usuario fue modificado", "Modificar usuario"); // Muestra un mensaje de éxito
}

```



PARCIAL 3 PRÁCTICA 2

Escena 1: En esta escena se puede leer una pequeña historia inicial, está el botón de sonido para escuchar una voz con la narración y el botón de pausa para detener el sonido. El botón de play en el centro para avanzar a la siguiente escena.



UI:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

// Script de Unity (1 referencia de recurso) 0 referencias
public class UI : MonoBehaviour
{
    public AudioSource voz; // Variable pública para asignar desde el Inspector el componente que reproduce el audio

    // Message de Unity (0 referencias)
    void Start()
    {
    }

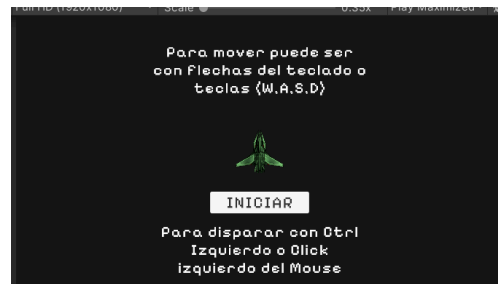
    // Message de Unity (0 referencias)
    void Update()
    {
    }

    // 0 referencias
    public void Reproducir()
    {
        voz.Play(); // Inicia la reproducción del clip de audio asignado al AudioSource
    }

    // 0 referencias
    public void Detener()
    {
        voz.Stop(); // Detiene por completo la reproducción actual del AudioSource
    }

    // 0 referencias
    public void CambioEscena()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1); // Carga la siguiente escena en la lista de Build Settings sumando 1 al índice de la escena activa
    }
}
```

Escena 2: En esta escena se muestra un canvas con Información sobre los controles del juego de la nave, como moverse y como disparar. Además de la visualización del personaje en el centro y un botón de “Iniciar” para avanzar a la siguiente escena.



CambioEscena:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class CambioEscena : MonoBehaviour
{
    // Message de Unity | 0 referencias
    void Start()
    {
    }

    // Message de Unity | 0 referencias
    void Update()
    {
    }

    // 0 referencias
    public void CambiarEscena()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
    }
}
```

Escena 3: Esta es la escena donde esta el juego en sí, donde empieza el nivel y mueves una nave con la que disparas con 1 arma, cuentas con 6 vidas y debes destruir naves enemigas sin que te quiten todas las vidas, haciendo la mayor cantidad de puntos por cada enemigo eliminado. Hay un poder que si lo obtienes te da una mejora de 2 armas extra durante un tiempo determinado para disparar más. Existen 3 tipos de naves enemigas, una es grande y lenta la cual solamente baja en línea recta, una que es más pequeña y más rápido que va directamente hacia el jugador, y una que igual se mueve hacia el jugador pero que dispara balas y se detiene solo apunta a cierta distancia. Cuando pierdes las vidas sale un panel que muestra tu puntuación y la opción de escribir un nombre para guardarlo todo en un archivo de excel. La frecuencia de aparición de los enemigos aumenta mientras más puntos ganes para aumentar la dificultad.





Mover:

```

public class mover : MonoBehaviour
{
    public AudioSource alarma;
    public AudioSource explosion;
    public GameObject efectoBala;
    public GameObject efectoCorazon;
    public Image corazonUI;

    Rigidbody2D nave;
    float x, y;
    public float velocidad = 10.0f;
    public float velocidadrotacion = 10.0f;

    private Camera camera;
    public float margen = 0.5f;

    SpriteRenderer sr;
    Color original;
    Color corazonColor;

    // Mensaje de Unity (2 referencias)
    void Start()
    {
        sr = GetComponent<SpriteRenderer>();
        nave = GetComponent<Rigidbody2D>();
        camera = Camera.main; // Obtiene la referencia a la cámara principal de la escena

        original = sr.color; // Guarda el color original de la nave
        corazonColor = corazonUI.color; // Guarda el color original del icono de corazón en la UI
    }

    // Mensaje de Unity (2 referencias)
    void Update()
    {
        float alto = camera.orthographicSize; // Obtiene la altura vertical del área visible de la cámara
        float ancho = alto * camera.aspect; // Calcula el ancho horizontal basado en el aspecto de la cámara
        Vector2 pos = transform.position;

        pos.x = Mathf.Clamp(pos.x, -ancho + margen, ancho - margen); // Limita la posición X para que no salga de la pantalla
        pos.y = Mathf.Clamp(pos.y, -alto + margen, alto - margen); // Limita la posición Y para que no salga de la pantalla
        transform.position = pos; // Aplica la posición restringida a la nave

        x = Input.GetAxis("Horizontal"); // Obtiene la entrada para rotación (teclas A/D o flechas)
        y = Input.GetAxis("Vertical"); // Obtiene la entrada para movimiento (teclas W/S o flechas)

        nave.velocity = transform.up * velocidad * y; // Mueve la nave hacia adelante según su rotación actual
        nave.angularVelocity = -x * velocidadrotacion; // Aplica la rotación a la nave basándose en la entrada horizontal
    }

    // Mensaje de Unity (2 referencias)
    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("enemigo") || collision.CompareTag("bala")) // Detecta colisión con enemigos o proyectiles
        {
            StartCoroutine(parpadeo()); // Inicia el efecto visual de recibir daño

            Vector2 pos = Vector2.Lerp(collision.transform.position, transform.position, 0.5f); // Calcula posición intermedia para el efecto
            GameObject Detonacion = Instantiate(efectoBala, pos, Quaternion.identity); // Crea el efecto visual de impacto
            Destroy(Detonacion, 2f); // Elimina el efecto de impacto tras 2 segundos

            GameObject CorazonCien = Instantiate(efectoCorazon, new Vector2(40.5f, 48.5f), Quaternion.identity); // Crea efecto en la UI
            Destroy(CorazonCien, 2f);

            alarma.Play(); // Reproduce el sonido de alerta
            explosion.Play(); // Reproduce el sonido de explosión
            CorazonCien.SetActive(false); // Oculta la UI de corazón para reducir el contador de vidas
            Destroy(collision.gameObject); // Destruye el objeto con el que se colisionó
        }
    }

    // Referencia
    IEnumerator parpadeo()
    {
        for (int i = 0; i < 5; i++) // Repite el ciclo de parpadeo 5 veces
        {
            corazonUI.color = Color.black; // Cambia el color del corazón a negro
            sr.color = Color.red; // Cambia el color de la nave a rojo
            yield return new WaitForSeconds(0.3f); // Espera 0.3 segundos
            corazonUI.color = corazonColor; // Restaura el color original del corazón
            sr.color = original; // Restaura el color original de la nave
            yield return new WaitForSeconds(0.3f);
        }
    }
}

```

Bala:

```

// Script de Unity (2 referencias de MonoBehaviour y MonoBehaviour)
public class Bala : MonoBehaviour
{
    public float velocidad = 10.0f; // Velocidad a la que saldrán disparadas las balas
    public GameObject bal; // Referencia al Prefab de la bala que se va a clonar

    //public Transform arreal;
    public Transform arreal;
    public Transform arreal2;

    public Transform[] posiciones; // Array de posiciones de disparo para el modo normal
    public Transform[] posiciones2; // Array de posiciones de disparo para el modo poder
    public AudioSource audioBala; // Sonido que se reproduce al disparar

    public Sprite spriteNormal; // Imagen de la nave en estado normal
    public Sprite spritePoder; // Imagen de la nave en modo poder

    private SpriteRenderer sr; // Referencia al componente que dibuja el sprite en pantalla

    public float duracion = 6.0f; // Tiempo que dura activo el efecto del power-up
    public bool modoPersonaje = false; // Booleano para alternar entre el modo normal y modo poder

    public AudioSource powerUp; // Sonido al recoger un power-up
    public GameObject poderEfecto; // Efecto visual al activar el poder
    public GameObject normalEfecto; // Efecto visual al regresar al modo normal

    // Mensaje de Unity (2 referencias)
    void Start()
    {
        audioBala.Stop(); // Asegura que el sonido de bala no suene al iniciar
        sr = GetComponent<SpriteRenderer>(); // Obtiene el componente SpriteRenderer de este objeto
        ActivarArma(); // Inicializa la configuración de las armas
    }

    // Mensaje de Unity (2 referencias)
    void Update()
    {
        if (Input.GetButtonDown("Fire1") && Time.timeScale != 0) // Detecta el clic de disparo siempre que el juego no esté pausado
        {
            //Disparar();
            Shoot(); // Ejecuta el método para crear las balas
            audioBala.Play(); // Reproduce el efecto de sonido del disparo
        }
    }

    // Referencia
    private void Shoot()
    {
        Transform[] actuales = modoPersonaje ? posiciones2 : posiciones; // Elige qué conjunto de posiciones usar según el modo activo
        foreach (Transform t in actuales) // Recorre todas las posiciones de disparo definidas
        {
            GameObject clonBala = Instantiate(bal, t.position, t.rotation); // Crea una instancia de la bala en la posición del objeto 't'
            Rigidbody2D RB = clonBala.GetComponent<Rigidbody2D>(); // Obtiene el componente físico de la bala recién creada
            RB.velocity = t.up * velocidad; // Aplica la fuerza de empuje hacia adelante del arma
            Destroy(clonBala, 1.5f); // Elimina la bala tras 1.5 segundos para no consumir memoria
        }
    }

    //private void Disparar()
    {
        GameObject clonBala1 = Instantiate(bal, arreal.position, arreal.rotation);
        Rigidbody2D cuerpoBala1 = clonBala1.GetComponent<Rigidbody2D>();
        cuerpoBala1.velocity = arreal.up * velocidad;
        Destroy(clonBala1, 1f);

        GameObject clonBala2 = Instantiate(bal, arreal2.position, arreal2.rotation);
        Rigidbody2D cuerpoBala2 = clonBala2.GetComponent<Rigidbody2D>();
        cuerpoBala2.velocity = arreal2.up * velocidad;
        Destroy(clonBala2, 1f);
    }

    // Referencia
    private void ActivarArma()
    {
        modoPersonaje = false; // Cambia al modo normal
        sr.sprite = spriteNormal; // Asigna el sprite de la nave normal
    }
}

```



```

@ Mensaje de Unity | 0 referencias
private void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.CompareTag("poder")) // Si toca un objeto con la etiqueta "poder"
    {
        GameObject clonEfecto = Instantiate(PoderEfecto, transform.position, Quaternion.identity); // Crea el efecto de activar poder
        Destroy(clonEfecto, 1f); // Limpia el efecto visual
        powerup.Play(); // Reproduce el sonido de recolección
        ActivarPoder(duracion); // Inicia el temporizador del modo poder
        Destroy(collision.gameObject); // Elimina el ítem del power-up del escenario
    }
}

1 referencia
public void ActivarPoder(float duracion)
{
    StopAllCoroutines(); // Detiene cualquier temporizador de poder previo
    StartCoroutine(PoderPersonaje(duracion)); // Inicia la rutina de tiempo para el nuevo modo poder
}

1 referencia
IEnumerator PoderPersonaje(float duracion)
{
    modopersonaje = true; // Activa el modo poder
    sr.sprite = spritePoder; // Cambia el sprite al del modo poder
    yield return new WaitForSeconds(duracion); // Espera el tiempo especificado mientras el poder está activo
    GameObject clonEfectoNormal = Instantiate(NormalEfecto, transform.position, Quaternion.identity); // Crea efecto al volver a la normalidad
    Destroy(clonEfectoNormal, 1f); // Limpia el efecto visual
    sr.sprite = spriteNormal; // Regresa el sprite a la normalidad
    modopersonaje = false; // Desactiva el modo poder
}

```

AudioFondo:

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

@ Script de Unity (1 referencia de recurso) | 0 referencias
public class AudioFondo : MonoBehaviour
{
    public AudioSource musica;
    @ Mensaje de Unity | 0 referencias
    void Start()
    {
        musica.Play();
    }

    @ Mensaje de Unity | 0 referencias
    void Update()
    {
    }
}

```

CrearPoderes:

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

@ Script de Unity (1 referencia de recurso) | 0 referencias
public class CrearPoderes : MonoBehaviour
{
    public GameObject poder; // Prefab del objeto power-up que se va a generar
    public float tiempo = 5.0f; // Tiempo en segundos entre cada aparición

    @ Mensaje de Unity | 0 referencias
    void Start()
    {
        StartCoroutine(GenerarPoder()); // Inicia la rutina que generará poderes en bucle
    }

    1 referencia
    IEnumerator GenerarPoder()
    {
        while (true) // Bucle infinito para crear poderes constantemente
        {
            yield return new WaitForSeconds(tiempo); // Espera el tiempo configurado antes de crear el siguiente
            crear(); // Llama al método que instancia el objeto
        }
    }

    1 referencia
    private void crear()
    {
        if (poder == null) // Verifica que se haya asignado un prefab antes de intentar instanciarlo
            return;

        Camera cam = Camera.main; // Obtiene la referencia a la cámara principal
        float alto = cam.orthographicSize; // Obtiene la altura del área visible
        float ancho = alto * cam.aspect; // Calcula el ancho visible basándose en el aspecto de pantalla

        float x = Random.Range(-ancho, ancho); // Genera una posición horizontal aleatoria dentro de los límites visibles
        //float y = (Random.value * 0.5f) * alto + 1 - alto * 1; // Define el punto de aparición inicial (fuera de pantalla arriba)
        Vector3 posicion = new Vector3(x, 60, 0);

        GameObject clon = Instantiate(poder, posicion, Quaternion.identity); // Crea una copia del objeto poder en la posición calculada
        Destroy(clon, 8f); // Elimina el objeto si no es recogido en 8 segundos para limpiar memoria
    }

    @ Mensaje de Unity | 0 referencias
    void Update()
    {
    }
}

```



CreadorEnemigos:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Script de Unity | 0 referencias de recurso | 0 referencias
public class CreadorEnemigos : MonoBehaviour
{
    int puntos;
    int puntosAct;
    public int CambioDificultad = 20; // Puntos necesarios para que el juego se vuelva más difícil
    public GameObject enemigo; // Referencia al Prefab del enemigo a generar
    public float tiempo = 2.0f; // Tiempo inicial entre la creación de cada enemigo

    // Mensaje de Unity | 0 referencias
    void Start()
    {
        StartCoroutine(GenerarEnemigo()); // Inicia la rutina que crea enemigos de forma recurrente
    }

    // Referencia
    IEnumerator GenerarEnemigo()
    {
        while (true) // Bucle infinito para seguir creando enemigos mientras el juego corra
        {
            yield return new WaitForSeconds(tiempo); // Espera el tiempo configurado antes de crear el siguiente
            crear(); // Llama al método de instanciación
        }
    }

    // Referencia
    private void crear()
    {
        if (enemigo == null) // Verifica que el Prefab esté asignado para evitar errores
            return;

        Camera cam = Camera.main; // Obtiene la cámara principal
        float alto = cam.orthographicSize;
        float ancho = alto * cam.aspect;

        float x = Random.Range(-ancho, ancho); // Calcula una posición X aleatoria dentro del área visible
        float y = (Random.value * 0.5f) * alto + 1 - alto - 1; // Define la posición inicial arriba en el eje Y
        Vector3 posicion = new Vector3(x, 60, 0); // Define la posición inicial arriba en el eje Y
        GameObject clone = Instantiate(enemigo, posicion, Quaternion.Euler(0f, 0f, -180f)); // Crea el enemigo rotado 180 grados
    }

    // Mensaje de Unity | 0 referencias
    void Update()
    {
        puntos = Contador.ContadorDePuntos(); // Obtiene el puntaje actual del sistema de juego
        if (puntos - puntosAct >= CambioDificultad) // Verifica si se ha superado el umbral para aumentar la dificultad
        {
            puntosAct = puntos;
            if (tiempo > 1) // Actualiza el marcador de control
            {
                tiempo -= 0.5f; // Reduce el tiempo entre enemigos de forma rápida mientras sea mayor a 1s
            }
            else if (tiempo > 0.1f) // Reduce el tiempo de forma más fina cuando ya es muy rápido
            {
                tiempo -= 0.1f;
            }
        }
    }
}
```

Enemigo (enemigo 1):

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Script de Unity | 0 referencias de recurso | 0 referencias
public class Enemigo : MonoBehaviour
{
    public float velocidad = 10f; // Velocidad de movimiento del enemigo
    Vector3 posicion; // Almacena la posición inicial al aparecer
    public float distancia = 10f; // Distancia máxima que el enemigo puede recorrer antes de ser destruido

    // Mensaje de Unity | 0 referencias
    void Start()
    {
        posicion = transform.position; // Guarda la posición donde nace el enemigo
    }

    // Mensaje de Unity | 0 referencias
    void Update()
    {
        transform.Translate(Vector3.up * velocidad * Time.deltaTime); // Mueve el enemigo hacia adelante (su eje Y local) constantemente
        float inicio = Vector3.Distance(posicion, transform.position); // Calcula la distancia recorrida desde el punto de inicio
        inicio = Mathf.Abs(inicio); // Asegura que el valor de la distancia sea positivo
        if (inicio > distancia) // Comprueba si el enemigo superó su límite de movimiento
        {
            Destroy(gameObject); // Destruye al enemigo para liberar memoria
        }
        // Destroy(gameObject, 10f);
    }
}
```

EliminarEnemigos:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class EliminarEnemigos : MonoBehaviour
{
    public GameObject efecto; // Referencia al efecto visual que aparecerá al destruir al enemigo

    void Start()
    {
    }

    private void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.CompareTag("enemigo")) // Verifica si el objeto que entró en contacto tiene la etiqueta "enemigo"
        {
            Instantiate(efecto, collision.transform.position, Quaternion.identity); // Crea el efecto visual en la posición donde ocurrió el choque
            Contador.EliminarEnemigos(); // Llama al método estático para sumar el puntaje por la eliminación
            Destroy(collision.gameObject); // Elimina al enemigo del juego
            Destroy(gameObject); // Elimina el objeto actual (ej. la bala) tras el impacto
        }
    }

    // Mensaje de Unity | 0 referencias
    void Update()
    {
    }
}
```



SeguirPersonaje (enemigo 2):

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// Script de Unity (1 referencia de recurso) 0 referencias
public class SeguirPersonaje : MonoBehaviour
{
    public float velocidad = 30f; // Velocidad de movimiento del objeto hacia el objetivo
    //Vector2 posicion;
    //public float distancia = 10f;

    public Transform objetivo; // Referencia a la posición del objeto que se desea seguir
    float timer;

    // Mensaje de Unity 0 referencias
    void Start()
    {
        //posicion = transform.position;
        objetivo = GameObject.FindGameObjectWithTag("Player").transform; // Busca automáticamente al objeto etiquetado como "Player" para asignarlo como objetivo
    }

    // Mensaje de Unity 0 referencias
    void Update()
    {
        if (objetivo == null) return; // Evita errores si el jugador es destruido durante el juego

        Vector3 Direccion = (objetivo.position - transform.position).normalized; // Calcula el vector de dirección normalizado hacia el jugador
        transform.position += Direccion * velocidad * Time.deltaTime; // Mueve el objeto hacia el jugador basándose en su dirección y velocidad

        transform.up = Direccion; // Rota el objeto para que siempre mire hacia el jugador
        // Acumula el tiempo transcurrido en el frame
        //if (timer >= 1)
        //{
        //    timer = 0;
        //}

        //transform.Translate(Vector3.up * velocidad * Time.deltaTime);
        float inicio = Vector2.Distance(posicion, transform.position);
        inicio = Mathf.Abs(inicio);
        print(inicio);
        if (inicio > distancia)
        {
            Destroy(gameObject);
        }
        //Destroy(gameObject, 10f);
    }
}
```

Contador:

```
// Script de Unity (1 referencia de recurso) 5 referencias
public class Contador : MonoBehaviour
{
    public GameObject ScorePanel; // Referencia al panel que se muestra al perder
    public TextMeshPro[] enemigos; // Texto en pantalla para mostrar los enemigos eliminados
    public TextMeshPro[] vidas; // Texto en pantalla para mostrar las vidas restantes
    public TextMeshPro[] PuntosFinales; // Texto para mostrar el puntaje final en el panel
    public TMP_InputField nombre; // Campo de texto para que el usuario ingrese su nombre

    public static int contadorEnemigos = 0; // Variable global que almacena el total de enemigos eliminados
    public static int vidasPersonaje = 6; // Variable global que almacena la cantidad de vidas

    string nom = "Jugador"; // Nombre por defecto si el usuario no ingresa uno

    // Mensaje de Unity 0 referencias
    private void Awake()
    {
        //StreamReader leer;
        //if (File.Exists("Puntaje.csv"))
        //{
        //    leer = File.OpenText("Puntaje.csv");
        //    string datos = leer.ReadLine();
        //    string[] p = datos.Split(';');
        //    contadorEnemigos = int.Parse(p[1]);
        //}
        //leer.Close();

    }

    // 0 referencias
    public void GuardarArchivo()
    {
        if (nombre.text.Length <= 20) // Valida que el nombre no exceda los 20 caracteres
        {
            //File.Delete("Puntaje.csv");
            StreamWriter escribir;
            escribir = File.AppendText("Puntaje.csv");
            string ptosfin = contadorEnemigos.ToString();
            if (nombre.text != "") nom = nombre.text;
            escribir.WriteLine(nom + ";" + ptosfin);
            escribir.Close();
            ScorePanel.SetActive(false);
            Application.Quit(); // Cierra la aplicación
        }
    }

    // Mensaje de Unity 0 referencias
    void Start()
    {
    }

    // 1 referencia
    public static void DescartarVidas()
    {
        vidasPersonaje--;
        if (vidasPersonaje <= 0) // Reduce la cantidad de vidas en uno
        { // Si las vidas llegan a cero...
            Time.timeScale = 0; // Pausa el juego congelando el tiempo
        }
    }

    // 1 referencia
    public static void EliminarEnemigos()
    {
        contadorEnemigos++; // Incrementa el contador de enemigos eliminados
    }

    // 1 referencia
    public static int ContadorDePuntos()
    {
        return contadorEnemigos; // Devuelve el valor actual de enemigos eliminados
    }

    // Mensaje de Unity 0 referencias
    void Update()
    {
        if (vidasPersonaje <= 0) // Comprueba si el jugador perdió
        {
            ScorePanel.SetActive(true); // Activa el panel de fin de juego
            PuntosFinales.text = contadorEnemigos.ToString(); // Muestra el puntaje obtenido en el texto del panel
        }
        enemigo.text = Contador.contadorEnemigos.ToString(); // Actualiza el contador de enemigos en la interfaz
        vidas.text = Contador.vidasPersonaje.ToString(); // Actualiza el contador de vidas en la interfaz
    }
}
```



ExplosionSonido:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[Script de Unity (1 referencia de recurso) | 0 referencias]
public class ExplosionSonido : MonoBehaviour
{
    public AudioSource explosion;
    [Mensaje de Unity | 0 referencias]
    void Start()
    {
        explosion.Play();
        Destroy(gameObject, 2f);
    }
}
```

EnemigoDisparo:

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class EnemigoDisparo : MonoBehaviour
{
    public float velocidad = 30f; // Velocidad de movimiento del enemigo
    public float velocidadBala = 60f; // Velocidad de disparo de la bala
    public float tiempoDisparo = 1f; // Tiempo de espera entre cada disparo

    public Transform objetivo; // Referencia al transform del jugador
    float timer;

    public Transform arma; // Posición desde donde se disparará la bala
    public GameObject bala; // Prefab de la bala que se clonará

    void Start()
    {
        objetivo = GameObject.FindGameObjectWithTag("Player").transform; // Busca y asigna automáticamente al jugador como objetivo
        StartCoroutine(Disparar()); // Inicia la rutina de disparo automático
    }

    void Update()
    {
        if (objetivo == null) return; // Evita errores si el jugador desaparece

        float dist = Vector2.Distance(objetivo.position, transform.position); // Calcula la distancia actual hacia el jugador
        Vector3 Direccion = (objetivo.position - transform.position).normalized; // Calcula la dirección normalizada hacia el jugador
        if (dist > 60f) // Si está a más de 60 unidades de distancia...
        {
            transform.position += Direccion * velocidad * Time.deltaTime; // ...se acerca al jugador
        }
        transform.up = Direccion; // Rota el enemigo para que siempre apunte hacia el jugador
        timer += Time.deltaTime;
    }

    IEnumerator Disparar()
    {
        while (true) // Bucle infinito para disparar constantemente
        {
            yield return new WaitForSeconds(tiempoDisparo); // Espera el tiempo configurado antes de disparar
            GameObject clonBala = Instantiate(bala, arma.position, Quaternion.identity); // Crea una bala en la posición del arma

            Rigidbody2D RB = clonBala.GetComponent<Rigidbody2D>(); // Accede al componente físico de la bala
            RB.velocity = arma.up * velocidadBala; // Impulsa la bala en la dirección hacia donde apunta el arma

            Destroy(clonBala, 2f); // Destruye la bala a los 2 segundos para liberar memoria
        }
    }
}
```